

Table des matières

Compilation	2
SizeOf	3
Headers/Préprocesseur	3
Code/Permutation	4
Opérateurs	4
Code/LigneDeCommande/GrandPetit	5
Code/ LigneDeCommande/Fichier	6
Code/String	6
Code/ListeChaine	7
Makefile	8
Code/Struct/Union	9
Code/Fibonacci	10
Code/Fonction/GrandPetit	10
Code/Conversion	11
Fichier/MotUnique&	11
Code/Repertoire	12
Code/Fichier/String	12
Code/Tri	13
Code/Power	14
AnalyseDeCode	14
Code/Binaire/Masque	15
Code/Struct/Enum	15
Code/Fichier/Mots	16
Headers	17
Compilation/Etapes	17
Code/Fichier/Compte&	17
Code/Fichier/ChercherPhrase&	18
Code/Fonction/Aire	18
Code/Boucle	19
Code	19
Code/Fibonacci/Rekursif	19
Code/String/Copy&	20
Code/String/Copy&	20
Code/Dossier	21
AnalyseCode	21
Iteratif/Rekursif	22

Compilation

Répondez aux questions suivantes :

- a. C est un langage compilé. Pourquoi ? Justifiez votre réponse.
- b. Quels sont les objectifs des options `-E`, `-o` et `-c` du compilateur **gcc** ?
Donnez un exemple d'utilisation pour chaque option.
- c. Quel code est implémenté dans le fichier `.h`
- d. Quel code est implémenté dans le fichier `.c`?

(2 points)

a)

C est un langage compilé car il est nécessaire de compiler le code avant de l'exécuter. En effet, sur linux, on doit d'abord faire la commande `gcc` avant d'exécuter le programme. En faisant cela on crée un fichier en binaire correspondant à notre code que notre ordinateur peut lire.

Langages compilés : C, C++...

Langages interprétés : Python, PHP...

b)

`-E` : permet de faire un appel du préprocesseur mais n'effectue pas la compilation.

Ex : `gcc -E bonjour.c`

`-o` : Permet de donner un nom au fichier généré lors de la compilation.

Ex : `gcc -o bonjour bonjour.c`

`-c` Appelle le préprocesseur, effectue la compilation et l'assemblage, mais ne fait pas l'édition de lien.

Seuls les fichiers objet (« `.o` ») sont générés. =>

Ex : `gcc -c bonjour.c`

c)

Le code présent dans le fichier `.h` correspond aux headers, c'est des fichiers qui contiennent les prototypes des fonctions. On définit aussi les structures

d)

Le code présent dans le fichier `.c` correspond au code source, contient toutes les fonctions nécessaires à la bonne exécution du code, ainsi que les librairies et fonction au début du code via `include`.

SizeOf

Vous êtes sur une machine de 64 bits et vous utilisez également un système d'exploitation 64bits. Considérez les variables suivantes :

```
char message1[10] = "Bonjour" ;  
char *message2 = "Bonjour" ;
```

Quelle est la valeur renvoyée par chacune des lignes suivantes ?

- a) sizeof (int)
- b) sizeof (int *)
- c) sizeof (char **)
- d) sizeof (float **)
- e) sizeof (message1)
- f) strlen(message1)
- g) sizeof (message2)
- h) strlen(message2)

Justifiez votre réponse. (2 points)

- a) 4 car les int sont défini sur 4 octets
- b) 8 car int * est un pointeur et les pointeurs sont défini sur 8 octets
- c) 8 car char ** est un pointeur de pointeur, donc c'est un pointeur.
- d) 8 pour la même raison qu'au dessus : float ** est un pointeur
- e) 10 car char est de taille 1 mais message est un tableau de 10 char donc 10x1=10.
- f) 7 car il y a 7 caractères dans "Bonjour"
- g) 8 car message 2 correspond à un pointeur.
- h) 7 car il y a 7 éléments dans "Bonjour"

Headers/Préprocesseur

Le contenu d'un fichier `opérateurs.h` est donné ci-dessous

```
#ifndef __OPERATEURS_H__  
#define __OPERATEURS_H__  
int num = 20;  
int add( int, int);  
#endif //__OPERATEURS_H__
```

Pourquoi utilisons-nous les directives du préprocesseur comme `ifndef`, `define` et `endif`? Justifiez votre réponse avec des exemples. (2 points)

Ces directives sont intéressantes lorsqu'il y a différents fichiers qui sont liés. En effet, on ne sait pas forcément quelles fonction ou librairies ont été défini dans les autres fonctions. On s'assure alors de ne pas définir 2 fois opérateurs et on s'assure de définir opérateurs dans le cas où il n'est pas défini.

Code/Permutation

Ecrivez un code complet en C dans le but de permuter les valeurs de deux variables. Votre code doit contenir les fichiers d'en-tête nécessaires, une fonction `échange(...)` et une fonction `main`. Le but de la fonction `échange(...)` est d'assurer la permutation des valeurs de deux nombres à virgule flottante. La fonction `main` doit appeler la fonction `échange(...)` et doit afficher les valeurs avant et après l'appel de la fonction. **(2 points)**

```
Include<stdio.h>
Include "échange.h"

Float *échange( float a, float b){
    Float *Tab[2];
    Tab[0]=&b
    Tab[1]=&a
    Return Tab;
}

Int main(){
    Float a = 3.5 ;
    Float b = 9.9 ;
    Printf('Avant échange : \na : %f b : %f'', a, b) ;
    Float *res = échange(a,b)
    Printf('Après échange : \na : %f b : %f'', *res, *(res+1)) ;
    Return 0 ;
}
```

Opérateurs

Quelle est la valeur renvoyée par chacune des lignes suivantes ?

- a) `8 % 4`
- b) `8 & 4`
- c) `8 | 4`
- d) `8 && 4`
- e) `8 || 4`
- f) `8 << 4`
- g) `8 / 4`
- h) `8 ** 4`

Justifiez votre réponse. (2 points)

- a) 0 car le reste de la division euclidienne de 8 par 4 est 0
- b) 0 car $8=1000b$ et $4=0100b$ donc $8\&4=0000b$
- c) 12 car $8=1000b$ et $4=0100b$ donc $8|4=1100b$
- d) 1 car $8\&4$ vaut toujours 1 en logique
- e) 1 car $8||4$ vaut toujours 1 en logique
- f) 128 car on decale (de 4 bits vers la gauche) les bits de $8=1000b$ et $128=10000000b$
- g) 2 car $8/4 = 2$
- h) Erreur ?

Code/LigneDeCommande/GrandPetit

Codez en C un programme qui prend des numéros (un ou plusieurs) passés par la **ligne de commande**. L'objectif de code est de calculer

- a) Le nombre de nombres
- b) Le numéro le plus grand
- c) Le numéro le plus petit
- d) La moyenne

N'utilisez pas `scanf`. (2 points)

```
Int main(int argc, char*argv){
    If (argc>1){
        Int max = atoi(argv[1]) ;
        Int min = atoi(argv[1]) ;
        Int moy =;

        Int i ;
        For (i=1 ;i<argc-1;i++){
            If(atoi(argv[i])>max){
                Max= atoi(argv[i]);
            }
            If(atoi(argv[i])<min){
                Min= atoi(argv[i]);
            }
            Moy=moy+ atoi(argv[i]);
        }
        Moy=moy/(argc-1);
        Printf("d% %d %d %d", argc-1, max, min, moy);
    }
}
```

Code/ LigneDeCommande/Fichier

Coder en C la fonction qui permet d'afficher les n premières et les n dernières lignes d'un fichier dont le nom de fichier et la valeur n sont passés par l'utilisateur en utilisant la **ligne de commande**.

Pensez aux commandes `head` et `tail` en Linux dont l'objectif est d'afficher les n premières et les n dernières lignes d'un fichier. **(2 points)**

```
#!/ Pas d'utilisation de head et tail
Int main(int argc, char*argv){
    If (argc != 3){
        Return 0 ;
    }
    FILE* fichier ;
    Fichier = fopen(argv[1], r) ;
    Char chaine[100] ;
    Char c ;
    Int n=atoi(argv[2]);
    Int i=0;
    Int nbLigne=1;

    While (c=fgetc(fichier)!=NULL){
        If(c=='\n'){
            nbLigne++;
        }
    }

    While (fgets(chaine, 100,fichier)!=NULL){
        If (i<n||i>=nbLigne-n){
            Printf("%s",chaine);
        }
        i++;
    }
}
```

Code/String

Sans utiliser les bibliothèques standards ou externes (par exemple `string.h`), écrivez le code en C qui

1. Calcule le nombre de caractères dans une autre chaine de caractères
2. Copie une chaine de caractères dans une autre chaine de caractères
3. Concatène deux chaines de caractères
4. Compare deux chaines de caractères.

Vous devez écrire des fonctions pour chacune de ces opérations sur les chaînes de caractères. **(2 points)**

```
Int calcule(char char1){
    Int i=0;
    While (char1[i] !='\0'){
        char1++;
    }
}
```

```

        i++;
    }
    Return i;
}

Char * Copie(char char1){
    Int n = Calcule(char1);
    Char Char2[n];
    Int I;
    For( i=0;i<n;i++){
        Char2[i]=char1[i];
    }
    Return &Char2;
}

Char *Concat(char char1, char char2){
    Int n1=calcule(char1) ;
    Int n2=calcule(char2) ;
    Char *Char3[n1+n2] ;
    Char3=copie(char1);
    Int i;
    For( i=n1;i<n2;i++){
        Char3[i]=&char2[i-n1];
    }
    Return char3
}

Int compare(char char1, char char2){
    Int n1=calcule(char1) ;
    Int n2=calcule(char2) ;
    If(n1 !=n2){
        Return 0 ;
    }
    Int i ;
    For(i=0 ;i<n1;i++){
        If(char1[i]!=char2[i]){
            Return 0
        }
    }
    Return 1;
}

```

Code/ListeChaine

En utilisant les listes génériques **simplement** chaînées, écrivez le code `liste.c` (et `liste.h`) et implémentez les fonctions suivantes :

1. **void inserer(list *, void*)** prend deux entrées : une liste d'entiers simplement chaînée et un pointeur générique. L'objectif de cette fonction est d'ajouter l'entier à la fin de la liste chaînée.
2. **void* supprimer (list *)** prend une entrée : une liste d'entiers simplement chaînée. L'objectif de cette fonction est de supprimer le premier élément dans la liste et de renvoyer cet élément en utilisant un pointeur générique.

Rappelez bien qu'une liste générique simplement chaînée fonctionne avec les différents types de données : les entiers, les nombres en flottant, les caractères ou même les structures. **(2 points)**

```
Typedef struct entier entier ;
Struct entier{
    Int val ;
    Char charact ;
    Float réel ;
    Struct list* next ;
} ;

Typedef struct list list ;
Struct list{
    entier* first ;
    entier* last
};

Void insérer(list* list, entier* entier){
    *(*list).last.next = entier;
    *(entier).next = *(list).first;
    *(list).last = entier
}

Int supprimer(list * list){
    Int retour = *(*list).first.valeur ;
    *(list).first = *(*list).first.next ;
    *(*list).last.next = *(list).first;
    Return retour;
}
```

Makefile

Quels sont les avantages à utiliser un *Makefile* et la commande *make*.

Vous avez des fichiers C suivants

: client.c, client.h, serveur.c, serveur.h, couleur.c et couleur.h. Ecrivez un Makefile qui peut générer deux fichiers exécutables appelés `server` et `client` en utilisant les fichiers précédents. L'exécutable `client` est généré en utilisant `client.c`, `client.h`, `couleur.h` et `couleur.c` et l'exécutable `server` est généré en utilisant les fichiers `serveur.c`, `serveur.h`, `couleur.h` et `couleur.c`. **(2 points)**

Un makefile et la commande make permettent d'éviter de recompiler tous les fichiers, les en-têtes et le code source en cas de changements mais uniquement les fichiers concernés. Cela permet aussi de compiler plusieurs fichiers en une ligne de commande.

```
CC ?= gcc
COBJS ?= client.o, couleur.o
SOBJS ?= serveur.o

.SUFFIXES: .c .o
```



```

SERVER = serveur
CLIENT = client

all: $(SERVER) $(CLIENT)

$(SERVER): $(SOBJS)
    $(CC) -o $(SERVER) $(SOBJS)

$(CLIENT): $(COBJS)
    $(CC) -o $(CLIENT) $(COBJS)

.c.o:
    $(CC) -c $.c

```

Code/Struct/Union

Quelles sont les différences entre les mots-clés **struct** et **union** en C ? Coder en C une structure de données **chanson** qui comporte les éléments suivants : identifiant, titre, nom de chanteur(s)/chanteuse(s), durée de la chanson, année de la sortie, nom de compositeur/compositrice, prix. Instanciez une variable de cette structure (par exemple, les détails de votre chanson préférée). Pensez à utiliser les pointeurs. **(2 points)**

Un union est une concaténation d'éléments (pouvant être de type différents) et peut contenir qu'un seul élément à la fois. Cela peut permettre d'optimiser de la place.

Un struct permet de regrouper des éléments différents mais ayant des caractéristiques en commun.

```

Struct Chanson {
    Char identifiant[50];
    Char titre[50]
    Int annee;
    Int duree ;
    Union Artiste{
        Char chanteur[50];
        Char compositeur[50];
    } a ;
    Union Prix{
        int prix1;
        float prix2;
    } p ;
}

Int main(){
    Struct Chanson Chanson1;
    Chanson1.identifiant = "POP";
    Chanson1.titre = "WAP" ;
    Chanson1.a.chanteur = "Cardi B" ;
    Chanson1.duree= 258 ;
    Chanson1.annee = 2020 ;
    Chanson1.p.Prix1 = 69 ;
}

```

Code/Fibonacci

La suite de Fibonacci est une suite d'entiers définie comme suit:

- $U_0=1$
- $U_1=1$
- $U_n = U_{n-1} + U_{n-2}$

Codez en C la fonction **fibonacci(int n)**, qui affiche les n termes de la suite de Fibonacci $U_0, U_1, U_2, \dots, U_n$. Vous devrez utiliser les boucles (par exemple, for, while, do..while) pour cette question. Par exemple fibonacci(6) affiche

1 1 2 3 5 8 (2 points)

```
Int fibonacci(int n){
    Int Liste[n] ;
    Liste[0]=1 ;
    Liste[1]=1 ;
    Int i ;
    If (n==1){
        Printf("1");
    }
    If (n==2){
        Printf("1 1");
    }

    For(i=0 ;i<n;i++){
        If(i<2){
            Liste[i]=Liste[i-1]+Liste[i-2];
        }
        Printf("%d",Liste[i]);
    }
    Return 0;
}
```

Code/Fonction/GrandPetit

Codez en C une fonction qui prend des entiers (un ou plusieurs). L'objectif de cette fonction est de calculer et renvoyer les valeurs suivantes

- a) Le numéro plus grand
- b) Le numéro plus petit
- c) La moyenne

Pensez à utiliser les pointeurs pour renvoyer ces valeurs.(2 points)

```
float * fctn(int ListeNum[]){

    int n=sizeof(ListeNum)/sizeof(ListeNum[0]);
    float max = ListeNum[0];
```

```

float min = ListeNum[0];
float moy=0;

int i;
for(i=0;i<n;i++){
    if (ListeNum[i]>max){
        max=ListeNum[i];
    }
    if (ListeNum[i]<min){
        min=ListeNum[i];
    }
    moy=moy+ListeNum[i];
}

moy=moy/n;
float *Fin[3];
Fin[0]=&max;
Fin[1]=&min;
Fin[2]=&moy;
return *Fin;
}

```

Code/Conversion

Coder en C la fonction **int conversion (char * str)** qui permet de convertir une chaîne de caractères **str** à un nombre entier (**int**). La fonction **conversion** renvoie le nombre après la conversion. Vous pouvez utiliser les fonctions disponibles dans les bibliothèques standards ou vous pouvez écrire votre propre fonction. **(2 points)**

```

Int conversion(char *str){
    Int num ;
    Num=Atoi(str) ;
    Return num ;
}

//Pour un float
float conversion(char *str){
    float num ;
    Num=Atof(str) ;
    Return num ;
}

```

Fichier/MotUnique&

Codez en C la fonction qui affiche les mots uniques dans un fichier. Le nom du fichier est saisi par l'utilisateur. Par exemple, la sortie de votre programme peut ressembler à ceci:

- main
- int
- print
- float

A faire

(2 points)

Code/Repertoire

Codez en C le programme qui cherche un fichier dans un répertoire et les sous-répertoires. Le nom de fichier ainsi que le chemin de répertoire sont saisis par l'utilisateur. Le programme affiche 'oui' ou 'non'. (2 points)

```
void lire_dossier(char * Repertoire, char* fichierRech){

    DIR* rep = NULL;
    struct dirent* fichierLu = NULL;
    rep = opendir(Repertoire);

    if (rep == NULL){//Cas NULL
        printf("Dossier non trouvé");
        return;
    }
    else{
        while((fichierLu = readdir(rep))!=NULL){

            if((strcmp(fichierLu->d_name, ".") && (strcmp(fichierLu->d_name, ".."))){

                if (strcmp(fichierLu->d_name, fichierRech) == 0){
                    printf("oui");
                    return;
                }
            }
        }
        Printf('\nnon') ;
        return;
    }
}
```

Code/Fichier/String

Codez en C un programme qui cherche une phrase passée par l'utilisateur dans un fichier. Le nom du fichier est également saisi par l'utilisateur. Votre programme doit afficher les numéros de lignes où la phrase apparaît. Par exemple, si l'utilisateur cherche le mot float dans un fichier `calcul.e.c`, l'affichage de votre programme peut ressembler à ceci :

- Ligne 4
- Ligne 12

- ...

(2 points)

```
char analyse_fichier(char * nom_de_fichier, char * phrase){

    FILE* fichier;
    fichier = fopen(nom_de_fichier, "r");

    int indicephrase=0;
    int C;
    int cpt=0;
    int Ligne=0;

    while(1){
        C=fgetc(fichier);
        if (C==EOF){
            break;
        }
        if (C==phrase[indicephrase]){
            indicephrase++;
        }
        else{
            indicephrase=0;
        }

        if (indicephrase==strlen(phrase)){
            cpt++;
            indicephrase=0;
        }

        //Incrémentation de la ligne et affichage
        if (C=='\n'){
            Ligne++;
            if (cpt!=0){
                printf("Mot : %s, Ligne %d, Occurrence %d\n", phrase,
                    Ligne, cpt);
                cpt=0;
            }
        }
    }
    fclose(fichier);
}
```

Code/Tri

Codez en C un programme qui prend des numéros (un ou plusieurs) passés par la ligne de commande. L'objectif de code est de

- a) Trier les nombres par ordre croissant
- b) Trier les nombres par ordre décroissant

N'utilisez pas `scanf`. (2 points)

```
Int main(int argc, char*argv){
```

```

If (argc ==1){
    Return 0 ;
}
Int Liste[argc-1] ;
Int i ;
For(i=0 ;i<argc-1 ;i++){
    Liste[i]=atoi(argv[i+1]);
}
Int ListeC[argc-1], ListeD[argc-1]
memcpy(ListeC, Liste, argc-1);
memcpy(ListeD, Liste, argc-1);
Int j, k ;
Int tampon1, tampon 2;
for(k=0;k<=argc-2;k++){
    for(j=argc-2;j>=k+1;j--){
        tampon1 = ListeC[j-1];
        tampon2 = ListeC[j];
        if (ListeC[j]<ListeC[j-1]){
            ListeC[j-1]=tampon2;
            ListeC[j]=tampon1;
        }
        if (ListeD[j]>ListeD[j-1]){
            ListeC[j-1]=tampon2;
            ListeC[j]=tampon1;
        }
    }
}
}
}

```

Code/Power

Coder en C la fonction **puissance** qui prend deux entiers **a** et **b** et qui renvoie la valeur du nombre **a** élevé à une puissance **b**. Par exemple, puissance(2,3) renvoie la valeur 8. N'utilisez pas la fonction **power()** (math.h). (1 point)

```

Int puissance(int a, int b){
    Int c=1 ;
    Int i;
    For(i=0 ;i<b;i++){
        c=c*a;
    }
    Return c;
}

```

AnalyseDeCode

Ci-dessous vous avez une partie de code. Quel est l'objectif de ce code ? (1 point)

```
#include <stdio.h>
```

```

#include <string.h>
int main() {
    char phrase[100];
    while(1) {
        printf("Entrez une phrase: ") ;
        fgets(phrase, 100, stdin) ;
        phrase[strlen(phrase)-1] = '\0' ;
        if (strncmp(phrase, "FIN", 100) == 0) {
            break ;
        }
        int num[2];
        sscanf(phrase, "%d %d", &num[0], &num[1]);
        printf ("%d\n", num[0] & num[1]) ;
    }
    return(0) ;
}

```

L'utilisateur rentre une phrase en boucle via fgets jusqu'à ce qui rentre le mot FIN. A chaque boucle, le code affiche le résultat de l'opération binaire a&b (où a et b sont les 2 premiers entiers entrée par l'utilisateur)

Code/Binaire/Masque

Considérer un entier d (**int d ;**). Codez en C une fonction qui vérifie si les 8eme et 25eme bits de **gauche** sont 1 (en binaire). Si les deux bits sont 1, la fonction renvoie 1 sinon 0. **(1 point)**

```

Int bits(int d){
    Int Masque=10000000000000000010000000b;
    If ((d&Masque)==Masque){
        Return 1 ;
    }
    Return 0 ;
}

```

Code/Struct/Enum

Coder en C une structure de données **Etudiant** qui comporte les éléments suivants : Nom, Prénom, Adresse (rue, ville etc.), Filière (ETI, IRC ou CGP), Modules choisis (5 maximum), la note dans chaque module.

Coder en C une structure de données **Module** qui comporte les éléments suivants : Nom du module, responsable du module, les intervenants dans le module.

Mais nous avons une contrainte. Nous ne savons pas la taille maximum (ou le nombre de caractères) de nom, prénom, rue, ville, nom du module etc. C'est l'utilisateur qui doit saisir ces valeurs pendant l'exécution.

Pour les deux structures **Etudiant** et **Module**, n'oubliez pas d'utiliser les types **enum**, **struct** et/ou **union**. **(1.5 points)**

```

struct Module{
    char Nom[50] ;
    char Respo[50] ;
    char Intervenant[50] ;
}
char Nom=malloc(taillemax) ;
struct Etudiant{
    char nom[10];
    char prenom[10];
    char adresse ;
    enum Filiere{
        ETI, IRC, CGP
    }
    struct Module Modules[5] ;
}

int main(){
    struct Etudiant variable;
    int taillemax=10;
    char* Nom=malloc(taillemax*sizeof(char)) ;
    strcpy(Nom, ``0123456789``) ;

    scanf(``%s``,variable.Nom) ;
    printf(``%s``,variable.Nom) ;
}

```

Code/Fichier/Mots

Codez en C la fonction qui affiche le compte de différents **mots** dans un fichier. Le nom du fichier est saisi par l'utilisateur. Par exemple, la sortie de votre programme peut ressembler à ceci.

```

int : 24 fois
print : 34 fois
... (2 points)

```

```

#include <string.h> ;
int main(){
    int compteur ;
    char ptrCaractere ;
    char mot[100] ;
    char* ptrnomFichier=&nomFichier[0] ;
    printf(``Nom : ``) ;
    scanf(``%s``,ptrnomFichier) ;
    printf(``mot : ``)
    scanf(``%s``,mot) ;
    char* content=malloc(sizeof(mot));
    FILE* fichier = NULL ;
    fichierEtudiant=fopen(ptrnomFichier,'r') ;
    fgets(content, sizeof(mot), fichier) ;
}
OU
int main(int argc, char* argv[])
{
    char caractereFichier ;
    int indicePhrase=0 ;
    int compteurLigne=0 ;
    int ligne = 1 ;
}

```



```

int taille ;
int fichier=open(argv[2],O_RDONLY) ;
do { #lecture caractere
taille = read(fichier, &caractereFichier,1);
    while (caractereFichier==argv[1][indicePhrase]&&taille>=1)
    {
        indicePhrase++;

        if (argv[1][indicePhrase]=='\0'){
            compteurLigne++;
            break;
        }
        Taille=read(fichier,&caractereFichier,1);
    }
    indicePhrase=0 ;
}while(taille>=1) ;
close(fichier) ;
printf(« %s :%d »,argv[1],compteurLigne)
}

```

Headers

Une des bonnes pratiques de programmation en C est de diviser le code en plusieurs modules (module1, module2,etc...) Pour chaque module on associe un fichier .h (exemple module1.h) et un fichier .c (exemple module2.c).

Quel code est implémenté dans le fichier .h ?

Quel code est implémenté dans le fichier .c ? **(0.5 point)**

Le code présent dans le fichier .h correspond aux headers, c'est des fichiers qui contiennent les prototypes des fonctions.

Le code présent dans le fichier .c correspond au code source, contient toutes les fonctions nécessaires à la bonne exécution du code, ainsi que les librairies et fonction au début du code via include.

Compilation/Etapes

Quelles sont les différentes étapes de compilation d'un programme C en utilisant un compilateur comme **gcc**. **(1 point)**

Préprocesseur -> Analyse lexicale -> Analyse syntaxique -> Langage intermédiaire -> optimisation de code -> génération de code natif

Code/Fichier/Compte&

Codez en C le code qui affiche le compte de différents caractères (a-z, A-Z, 0-9) dans un fichier. Le nom du fichier est saisi par l'utilisateur. Par exemple, la sortie de

votre programme peut rassembler comme ci-dessous.

A : 25 fois

B : 3 fois

... (2 points)

A Faire

Code/Fichier/ChercherPhrase&

Codez en C un programme qui cherche une phrase (par exemple 'Bonjour le monde !') passée par l'utilisateur dans un fichier. Le nom du fichier est également saisi par l'utilisateur. Votre programme doit afficher les numéros de lignes où la phrase apparaît. Par exemple, si l'utilisateur cherche le mot 'Bonjour le monde !' dans un fichier message.txt, l'affichage de votre programme peut ressembler à ceci :

Ligne 123 Ligne 678 ... (2 points)

A Faire

Code/Fonction/Aire

Coder en C la fonction **aire** qui permet de calculer l'aire d'un rectangle en fonction de sa longueur et de sa largeur. Les entrées de la fonction **aire** sont : la longueur et la largeur du rectangle. La fonction **aire** renvoie la valeur de l'aire.

Coder en C la fonction **perimetre** qui permet de calculer le périmètre d'un rectangle en fonction de sa longueur et de sa largeur. Les entrées de la fonction **périmètre** sont : la longueur et la largeur du rectangle. La fonction **périmètre** renvoie la valeur du périmètre. (1 point)

```
Float aire(float long, float larg){  
    Return long*larg;  
}  
  
Float perim(float long, float larg){  
    Return 2*(long+larg);  
}
```

Code/Boucle

Quelle est la principale différence entre la boucle `while` et la boucle **`do..while`**?

Coder en C la fonction **`void afficher_1()`** qui permet d'afficher les nombres inférieurs ou égaux à 1000, qui sont divisibles par 5 et non divisibles par 3 en utilisant une boucle **`while`**.

Coder en C la fonction **`void afficher_2()`** qui permet d'afficher les nombres inférieurs ou égaux à 1000, qui sont divisibles par 5 et non divisibles par 3 en utilisant une boucle **`do..while`**. **(2 points)**

```
Void afficher_1(){  
  
}  
Void afficher_2(){  
  
}
```

Code

Coder en C la fonction **`char* conversion (int n)`** qui permet de convertir un entier `n` en une chaîne de caractères. La fonction `conversion` renvoie la chaîne de caractères résultante. Vous pouvez utiliser les fonctions disponibles dans les bibliothèques standards ou vous pouvez écrire votre propre fonction. **(1 point)**

```
Char* conversion(int n) {  
    Char * str[80];  
    sprintf(str, "%d", n);  
    return str ;  
}
```

Code/Fibonacci/Recuratif

La suite de Fibonacci est une suite d'entiers définie comme suit:

- $U_0=1$
- $U_1=1$
- $U_n = U_{n-1} + U_{n-2}$

En utilisant une méthode récursive, coder en C la fonction **`void fibonacci(int n)`** qui prend `n` comme entrée et affiche les `n` termes de la suite de Fibonacci $U_0, U_1, U_2, \dots, U_n$. **(1.5 points)**

```
int fibonacci_recuratif(int n){  
    int Un=1 ;  
    if (n==1 || n==0){
```

```

        Un=1 ;
    }
    else{
        Un= fibonacci_recuratif(n-1)+ fibonacci_recuratif(n-2) ;
    }
    printf(« %d »,Un) ;
    return Un ;
}

```

Code/String/Copy

Examinez le code C suivant :

```

char message1[] = "Bonjour";
char caractere1 = 'a' ;
int inum1 = 12 ;
float fnum2 = 24 ;
char message2[100] ;
char caractere2 ;
int inum2 ;
float fnum2 ;

```

Compléter le code précédent avec les instructions nécessaires pour

1. Copier message1 dans message2
2. Copier caractere1 dans caractere2
3. Copier inum1 dans inum2
4. Copier fnum1 dans fnum2

Vous pouvez utiliser les fonctions disponibles dans la bibliothèque <string.h>. **(1 point)**

```

Strcpy(message2,message1);
Caractere2=caractere1;
Inum2=inum2 ;
Fnum1=fnum2 ;

```

Code/String/Copy

Considérons la chaîne de caractères mon_message :

```

char mon_message[100] = "";

```

Coder en C les instructions nécessaires pour:

1. Copier "Bonjour" dans mon_message
2. Ajouter "Le monde" à la fin de mon_message
3. Afficher la taille de mon_message suite aux intructions précédentes

Vous pouvez utiliser les fonctions disponibles dans la bibliothèque <string.h>. **(2 points)**

```

mon_message [0] = 'B';
mon_message [1] = 'o';
mon_message [2] = 'n';
mon_message [3] = 'j';
mon_message [4] = 'o';
mon_message [5] = 'u';
mon_message [6] = 'r';
mon_message [7] = '\0';
Strcat(mon_message, "'Le monde'") ;
X= Strlen(mon_message) ;

```

Code/Dossier

Examiner le contenu des deux fichiers repertoire.h et repertoire.c. Compléter le code de la fonction **void lire_dossier(char* nom_dossier)** qui permet de parcourir le répertoire, dont le nom est passé en paramètre, et affiche les noms de tous les fichiers dans ce répertoire. **(1 point)**

```

//fichier repertoire.h

#ifndef REP_H
#define REP_H
    void lire_dossier(char*) ;
#endif

//fichier repertoire.c
#include "repertoire.h "
void lire_dossier(char * Dossier){

    DIR* rep = NULL;
    struct dirent* fichierLu = NULL; // Déclaration d'un pointeur vers la
    structure dirent.
    rep = opendir(Dossier);

    if (rep == NULL){//Cas NULL
        printf("Dossier non trouvé");
        return;
    }
    else{
        while((fichierLu = readdir(rep))!=NULL){//Parcours du dossier

            if((strcmp(fichierLu->d_name, ".")&&(strcmp(fichierLu->d_name, ".."))){ //Elimination des cas . et ..

                //Affichage Dossier et fichier
                printf("%s\n", fichierLu->d_name);

            }

        }
    }
    return;
}

```

AnalyseCode

Ci-dessous vous avez une partie de code. Quel est l'objectif de ce code?

```
int num ;
while(1) {
printf("Entrez un numéro: ") ;
scanf("%d", &num) ;
if (num > 3) {
break;
}
}
```

Le code demande à l'utilisateur un nombre > 3 . Si ce n'est pas le cas, il redemande un nouveau nombre.

(1 point)

Iteratif/Recuratif

Quelles sont les différences entre les fonctions récursives et les approches itératives. Donnez un exemple pour chaque approche. **(1 point)**

Récuratif : on appelle à nouveau la fonction dans la fonction

Itératif : in seul appel de la fonction (on utilise des boucles en conséquence)